

MA 226 Project 2

Michelle Megnauth, Sam Sherman, Lucas Garcia Valdivia

April 28, 2026

Contents

Group Member Contributions	3
1 Introduction	3
2 Pendulum Model	4
2.1 Derivation of the Pendulum ODE	4
2.2 Linearization via Small-Angle Approximation	4
2.2.1 General Solution	4
2.2.2 Particular Solution and Graph	5
2.3 Long-Term Behavior	5
2.4 Nonlinear System (Large Angles)	6
2.4.1 First-Order System	6
2.4.2 Euler Approximation	6
2.5 Critical Points and Phase Portrait	6
2.5.1 Classification	7
3 Population Dynamics Model	8
3.1 Interacting Population Model: Bacteria and Antibiotic Treatment Effect . .	8
3.2 Independent Growth Model	8
3.2.1 Model Justification	8
3.2.2 System of Equations and Critical Points	8
3.2.3 Interpretation	9
3.2.4 Euler Approximation	9
3.2.5 Phase Portrait	10
3.3 Predator-Prey Interaction Model	10
3.3.1 Model Description	10
3.3.2 Critical Points and Classification	10
3.3.3 Euler Approximation	11
3.3.4 Phase Portrait	11
3.3.5 Interpretation	12
3.4 Advantages and Limitations	12
A Julia Code for Part 1: Pendulum Phase Portrait	13
B Julia Code for Part 2: Population Dynamics Model	15

Group Member Contributions

Michelle Megnauth:

- Helped compute and classify critical points for the interaction system
- Contributed to the development and analysis of the independent population growth model (Verhulst and Gompertz)
- Wrote the interpretation and discussion of system dynamics
- Formatted and organized the full $\text{\LaTeX}$ report

Lucas Garcia Valdivia:

- Contributed to the development and analysis of the independent population growth model (Verhulst and Gompertz)
- Helped compute and classify critical points for the interaction system
- Helped compute and classify critical points for the independent system
- Assisted with interpretation of equilibrium behavior in the independent model

Sam Sherman:

- Completed Question 1 (pendulum model and analysis)

1 Introduction

In Project 2, we look at two main systems, a pendulum model coming from Standard Newtonian mechanics and a system of populations that is modeled using logistic, Gompertz, and predator-prey interaction.

2 Pendulum Model

2.1 Derivation of the Pendulum ODE

Consider a pendulum of length L under constant gravitational acceleration g . Let $\theta(t)$ denote the angle between the pendulum and the vertical.

The tangential acceleration is given by:

$$a = -g \sin(\theta)$$

Converting to angular acceleration using radius L :

$$\alpha = -\frac{g}{L} \sin(\theta)$$

Since angular acceleration is the second derivative of θ :

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L} \sin(\theta)$$

Rewriting:

$$\theta'' + \frac{g}{L} \sin(\theta) = 0$$

This is a nonlinear second-order ordinary differential equation.

2.2 Linearization via Small-Angle Approximation

For small angles, $\sin(\theta) \approx \theta$, giving:

$$\theta'' + \frac{g}{L} \theta = 0$$

2.2.1 General Solution

The general solution to this linear ODE is:

$$\theta(t) = A \cos(\omega t) + B \sin(\omega t)$$

$$\theta'(t) = -A\omega \sin(\omega t) + B\omega \cos(\omega t)$$

where:

$$\omega = \sqrt{\frac{g}{L}}$$

Applying initial conditions:

$$\theta(0) = \theta_0, \quad \theta'(0) = \omega_0$$

we obtain:

$$A = \theta_0, \quad B = \frac{\omega_0}{\omega}$$

Thus:

$$\theta(t) = \theta_0 \cos\left(\sqrt{\frac{g}{L}} t\right) + \frac{\omega_0}{\sqrt{\frac{g}{L}}} \sin\left(\sqrt{\frac{g}{L}} t\right)$$

2.2.2 Particular Solution and Graph

For:

$$g = 10, \quad L = 1, \quad \theta_0 = 0, \quad \omega_0 = \frac{\pi}{4}$$

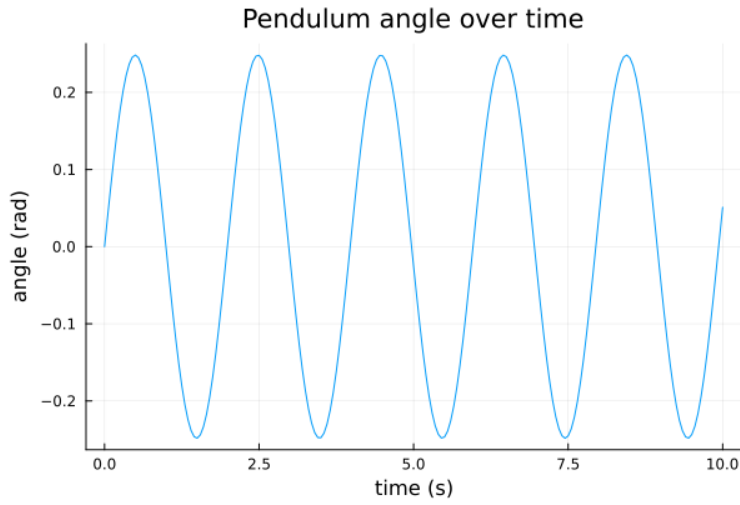


Figure 1: Solution to the linearized pendulum equation

2.3 Long-Term Behavior

For nontrivial initial conditions:

$$\theta(0) \notin \{k\pi \mid k \in \mathbb{Z}\}$$

Multiply the ODE by θ' :

$$\theta''\theta' + \frac{g}{L} \sin(\theta) \theta' = 0$$

This yields:

$$\frac{d}{dt} \left(\frac{1}{2} (\theta')^2 \right) + \frac{d}{dt} \left(\frac{g}{L} (1 - \cos \theta) \right) = 0$$

Thus:

$$\frac{1}{2} (\theta')^2 + \frac{g}{L} (1 - \cos \theta) = C$$

This represents conservation of energy. Therefore, nontrivial solutions oscillate and do not decay to zero.

2.4 Nonlinear System (Large Angles)

2.4.1 First-Order System

Let $\omega = \theta'$. Then:

$$\begin{cases} \theta' = \omega \\ \omega' = -\frac{g}{L} \sin(\theta) \end{cases}$$

2.4.2 Euler Approximation

The forward Euler method approximates the system for various initial conditions. Numerical error accumulates over time, which explains the observed growth in amplitude.

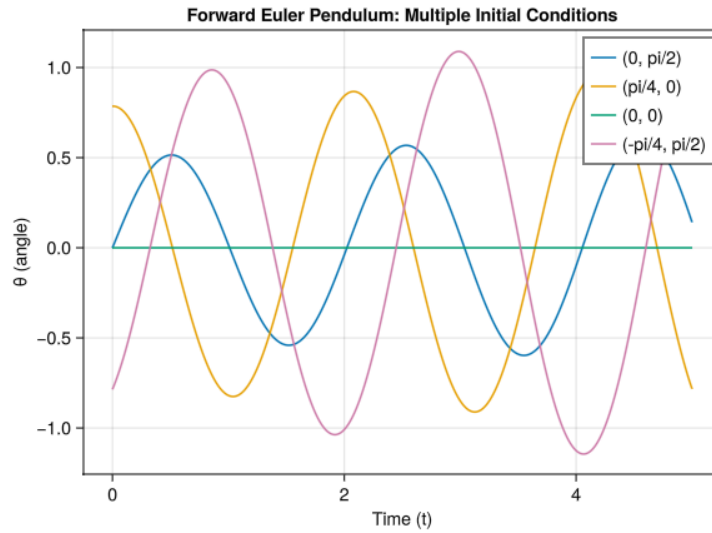


Figure 2: Forward Euler approximations for different initial conditions

2.5 Critical Points and Phase Portrait

Critical points satisfy:

$$\theta' = 0, \quad \omega' = 0$$

From $\theta' = \omega$:

$$\omega = 0$$

From $\omega' = -\frac{g}{L} \sin(\theta)$:

$$\sin(\theta) = 0 \Rightarrow \theta = k\pi$$

Thus:

$$(\theta, \omega) = (k\pi, 0), \quad k \in \mathbb{Z}$$

2.5.1 Classification

- If k is even ($\theta = 0, 2\pi, \dots$): center (stable oscillations)
- If k is odd ($\theta = \pi, 3\pi, \dots$): saddle (unstable equilibrium)

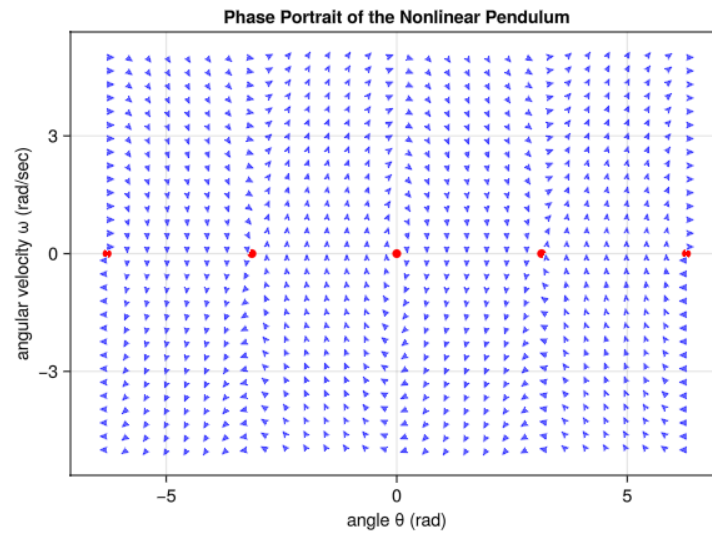


Figure 3: Phase portrait of the nonlinear pendulum

3 Population Dynamics Model

3.1 Interacting Population Model: Bacteria and Antibiotic Treatment Effect

To study how two interacting populations evolve over time, we modeled the interaction between a bacteria population and the effect of an antibiotic treatment as a dynamical system. Let $x(t)$ represent the bacteria population (in millions of cells), and let $y(t)$ represent the antibiotic treatment effect, measured on a normalized scale from 0 to 100. These variables are reasonable because bacteria naturally grow over time, while treatment effectiveness increases and eventually levels off due to biological limits.

3.2 Independent Growth Model

3.2.1 Model Justification

Here, we consider where these two quantities evolve independently of each other:

The bacteria population is modeled using the Verhulst (logistic) model:

$$x' = 0.8x \left(1 - \frac{x}{500}\right)$$

This reflects resource-limited growth, where the population approaches a carrying capacity of 500.

The antibiotic treatment effect is modeled using the Gompertz model:

$$y' = 0.6y \ln \left(\frac{100}{y}\right)$$

This captures how treatment effectiveness tends to increase quickly at first, then slows down as it approaches a maximum value. This behavior is consistent with how many biological treatments behave in actual practice.

3.2.2 System of Equations and Critical Points

The system is:

$$\begin{cases} x' = 0.8x \left(1 - \frac{x}{500}\right) \\ y' = 0.6y \ln \left(\frac{100}{y}\right) \end{cases}$$

Setting derivatives equal to zero for the critical points:

$$x = 0 \quad \text{or} \quad x = 500, \quad y = 100$$

Thus, the critical points are:

$$(0, 100), \quad (500, 100)$$

The Jacobian matrix is:

$$J(x, y) = \begin{bmatrix} 0.8 \left(1 - \frac{2x}{500}\right) & 0 \\ 0 & 0.6 \left(\ln\left(\frac{100}{y}\right) - 1\right) \end{bmatrix}$$

At $(0, 100)$:

$$\lambda_1 = 0.8 > 0, \quad \lambda_2 = -0.6 < 0$$

This is a saddle point since the eigenvalues have different signs.

At $(500, 100)$:

$$\lambda_1 = -0.8 < 0, \quad \lambda_2 = -0.6 < 0$$

This is a stable node since both of the eigenvalues are negative.

3.2.3 Interpretation

This shows that, over time, all trajectories converge to $(500, 100)$, meaning the bacteria approach carrying capacity while the treatment effect approaches its maximum, no matter the initial condition, assuming it is positive.

3.2.4 Euler Approximation

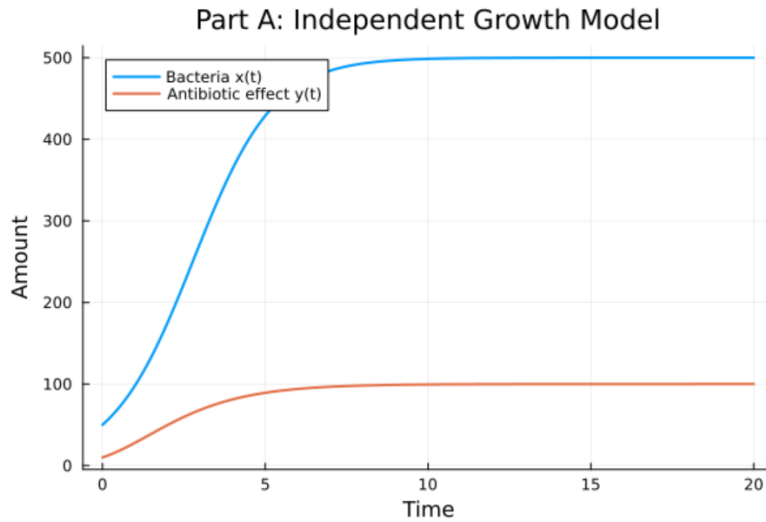


Figure 4: Part A: Independent Growth Model (Euler Approximation)

3.2.5 Phase Portrait

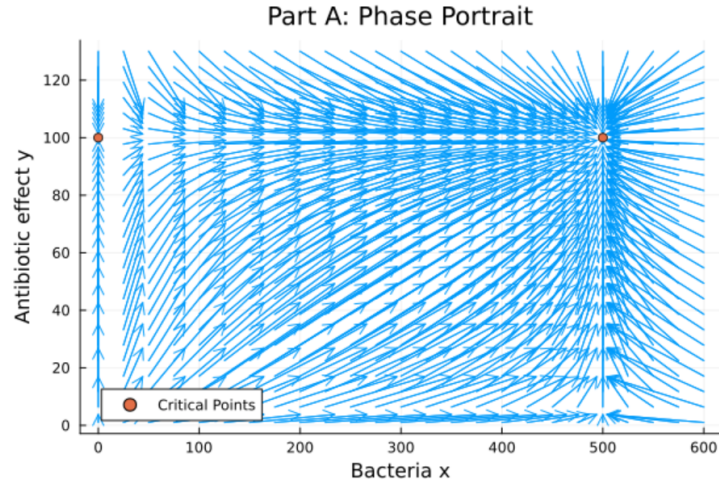


Figure 5: Part A: Phase Portrait of the Independent Growth Model

As shown in Figure 5, trajectories move directly toward the equilibrium, confirming the absence of interaction between variables.

3.3 Predator-Prey Interaction Model

3.3.1 Model Description

Now we extend the model and introduce interaction terms:

$$\begin{cases} x' = 0.8x \left(1 - \frac{x}{500}\right) - 0.002xy \\ y' = 0.6y \ln\left(\frac{100}{y}\right) + 0.001xy \end{cases}$$

Here, bacteria act as prey and antibiotic treatment acts as a suppressive predator. The interaction terms reflect that stronger treatment reduces bacteria, while a larger bacterial presence can increase treatment response.

3.3.2 Critical Points and Classification

Critical points of the system are:

$$(0, 100), \quad (295.46, 163.63)$$

At $(0, 100)$, the eigenvalues have opposite signs, so it is a saddle point.

At $(295.46, 163.63)$, eigenvalues are:

$$\lambda = -0.5364 \pm 0.3044i$$

Since the real part is negative, this is a stable spiral.

3.3.3 Euler Approximation

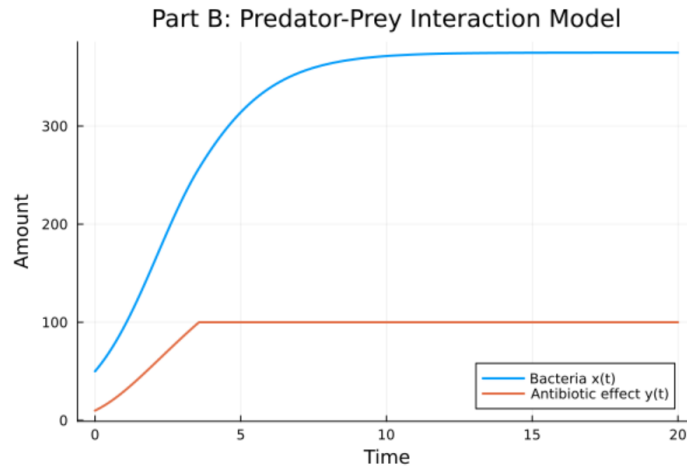


Figure 6: Part B: Predator-Prey Interaction Model (Euler Approximation)

3.3.4 Phase Portrait

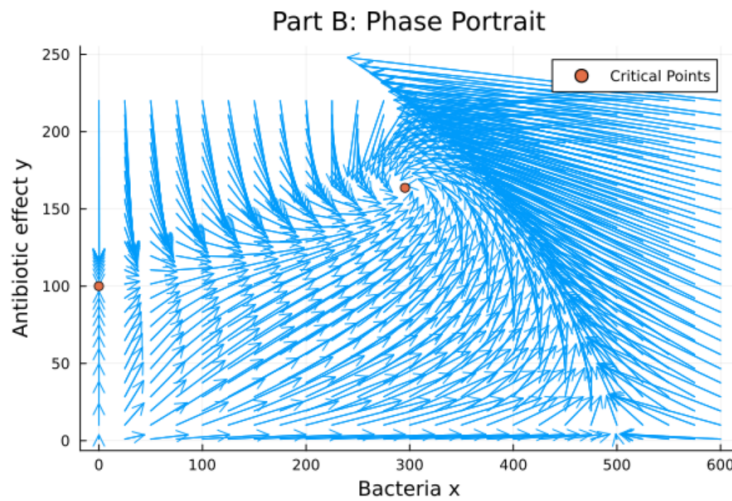


Figure 7: Part B: Phase Portrait of the Predator-Prey Interaction Model

As shown in Figure 7, the trajectories spiral inward, confirming the presence of a stable equilibrium.

3.3.5 Interpretation

The system exhibits oscillatory behavior that gradually spirals inward toward the stable equilibrium at $(295.46, 163.63)$. This reflects the feedback loop between bacteria and treatment as bacteria grow, treatment intensifies; as treatment intensifies, bacteria are suppressed, reducing the stimulus for further response. The cycle repeats with decreasing amplitude until the system settles. Notably, bacteria stabilize near 295, well below their carrying capacity of 500, confirming that the antibiotic suppresses but does not eliminate the population.

3.4 Advantages and Limitations

Advantages:

- This model is relatively simple and can be analyzed using standard tools.
- It captures important qualitative features of biological systems, including limited growth and saturation effects.

Limitations:

- The parameters allow equilibrium values outside biologically realistic ranges, capped at 100.
- The model does not include antibiotic decay over time.
- It also does not account for antibiotic resistance developing.
- The model also ignores immune system effects, which are a significant factor in real-life infections.

A Julia Code for Part 1: Pendulum Phase Portrait

```
using GLMakie

# parameters
g = 10
L = 1

# grid (theta, omega)
s = range(-2*pi, 2*pi, length=30)
ws = range(-5, 5, length=30)

# system definition
U(, w) = w
V(, w) = -(g/L) * sin()

# grid points
px = [ for in s, w in ws] |> vec
py = [w for in s, w in ws] |> vec

# vector field
u = [U(, w) for in s, w in ws] |> vec
v = [V(, w) for in s, w in ws] |> vec

# normalize arrows
mag = sqrt.(u.^2 .+ v.^2) .+ 1e-6

# plot
portrait, ax = arrows(
    px, py,
    u ./ mag, v ./ mag;
    lengthscale = 0.15,
```

```

        arrowsize = 10,
        color = (:blue, 0.6)
    )

# axis labels + title
ax.xlabel = "angle (rad)"
ax.ylabel = "angular velocity (rad/sec)"
ax.title = "Phase Portrait of the Nonlinear Pendulum"

# equilibrium points ( = k, w = 0)
for k in -2:2
    scatter!(ax, [k * pi], [0];
        color = :red, markersize = 10)
end

display(portrait)

```

B Julia Code for Part 2: Population Dynamics Model

```
import Pkg; Pkg.add("NLsolve")

using Plots
using LinearAlgebra
using NLsolve

# Bacteria and Antibiotic Treatment Effect

# Parameters
r = 0.8
K = 500.0

a = 0.6
M = 100.0

alpha = 0.002
beta = 0.001

# PART A: Independent Verhulst + Gompertz Model

function f_independent(x, y)
    dx = r*x*(1 - x/K)
    dy = a*y*log(M/y)
    return dx, dy
end

independent_points = [(0.0, M), (K, M)]

function jacobian_independent(x, y)
    return [
        r*(1 - 2*x/K)    0.0;
        0.0              a*(log(M/y) - 1)
    ]
end

println("PART A: Independent Model Critical Points")
for p in independent_points
```

```

x, y = p
J = jacobian_independent(x, y)
eigs = eigvals(J)

println("Critical point: ", p)
println("Eigenvalues: ", eigs)

if all(real.(eigs) .< 0)
    println("Classification: stable node")
elseif all(real.(eigs) .> 0)
    println("Classification: unstable node")
elseif prod(real.(eigs)) < 0
    println("Classification: saddle point")
else
    println("Classification: needs more analysis")
end

println()
end

# Euler method for independent model

function euler_independent(x0, y0, dt, tf)
    N = Int(round(tf/dt))
    t = collect(0:dt:tf)

    x = zeros(N+1)
    y = zeros(N+1)

    x[1] = x0
    y[1] = y0

    for n in 1:N
        dx, dy = f_independent(x[n], y[n])
        x[n+1] = x[n] + dt*dx
        y[n+1] = y[n] + dt*dy
    end

    return t, x, y
end

```



```

end

x0 = 50.0
y0 = 10.0
dt = 0.05
tf = 20.0

t1, x1, y1 = euler_independent(x0, y0, dt, tf)

println("PART A Euler Approximation at t = ", tf)
println("Bacteria x(tf) = ", x1[end])
println("Antibiotic effect y(tf) = ", y1[end])
println()

plot(t1, x1, label="Bacteria x(t)", linewidth=2)
plot!(t1, y1, label="Antibiotic effect y(t)", linewidth=2)
xlabel!("Time")
ylabel!("Amount")
title!("Part A: Independent Growth Model")
savefig("part_a_euler.png")

# Phase portrait for independent model

xvals = range(0.0, 600.0, length=25)
yvals = range(1.0, 130.0, length=25)

X = repeat(collect(xvals)', length(yvals), 1)
Y = repeat(collect(yvals), 1, length(xvals))

U = similar(X)
V = similar(Y)

for i in eachindex(X)
    U[i], V[i] = f_independent(X[i], Y[i])
end

quiver(
    vec(X), vec(Y),
    quiver=(vec(U), vec(V)),

```

```

    xlabel="Bacteria x",
    ylabel="Antibiotic effect y",
    title="Part A: Phase Portrait",
    label=false
)

scatter!([0.0, K], [M, M], label="Critical Points")
savefig("part_a_phase_portrait.png")

# PART B: Predator-Prey Interaction Model

function f_interaction(x, y)
    dx = r*x*(1 - x/K) - alpha*x*y
    dy = a*y*log(M/y) + beta*x*y
    return dx, dy
end

function jacobian_interaction(x, y)
    return [
        r*(1 - 2*x/K) - alpha*y      -alpha*x;
        beta*y                        a*(log(M/y) - 1) + beta*x
    ]
end

function equilibrium_system!(F, z)
    x = z[1]
    y = z[2]
    F[1] = r*x*(1 - x/K) - alpha*x*y
    F[2] = a*y*log(M/y) + beta*x*y
end

interaction_points = [(0.0, M)]

guess = [300.0, 160.0]
solution = nlsolve(equilibrium_system!, guess)
push!(interaction_points, (solution.zero[1], solution.zero[2]))

println("PART B: Predator-Prey Model Critical Points")
for p in interaction_points

```

```

x, y = p
J = jacobian_interaction(x, y)
eigs = eigvals(J)

println("Critical point: ", p)
println("Eigenvalues: ", eigs)

if all(real.(eigs) .< 0)
    if any(abs.(imag.(eigs)) .> 1e-6)
        println("Classification: stable spiral")
    else
        println("Classification: stable node")
    end
elseif all(real.(eigs) .> 0)
    if any(abs.(imag.(eigs)) .> 1e-6)
        println("Classification: unstable spiral")
    else
        println("Classification: unstable node")
    end
elseif prod(real.(eigs)) < 0
    println("Classification: saddle point")
else
    println("Classification: needs more analysis")
end

println()
end

# Euler method for interaction model

function euler_interaction(x0, y0, dt, tf)
    N = Int(round(tf/dt))
    t = collect(0:dt:tf)

    x = zeros(N+1)
    y = zeros(N+1)

    x[1] = x0
    y[1] = y0

```

```

    for n in 1:N
        dx, dy = f_interaction(x[n], y[n])
        x[n+1] = x[n] + dt*dx
        y[n+1] = y[n] + dt*dy
        x[n+1] = max(x[n+1], 0.0)
        y[n+1] = max(y[n+1], 0.001)
        y[n+1] = min(y[n+1], 100.0)
    end

    return t, x, y
end

dt2 = 0.01
tf2 = 20.0

t2, x2, y2 = euler_interaction(x0, y0, dt2, tf2)

println("PART B Euler Approximation at t = ", tf2)
println("Bacteria x(tf) = ", x2[end])
println("Antibiotic effect y(tf) = ", y2[end])
println()

plot(t2, x2, label="Bacteria x(t)", linewidth=2)
plot!(t2, y2, label="Antibiotic effect y(t)", linewidth=2)
xlabel!("Time")
ylabel!("Amount")
title!("Part B: Predator-Prey Interaction Model")
savefig("part_b_euler.png")

# Phase portrait for interaction model

xvals2 = range(0.0, 600.0, length=25)
yvals2 = range(1.0, 220.0, length=25)

X2 = repeat(collect(xvals2)', length(yvals2), 1)
Y2 = repeat(collect(yvals2), 1, length(xvals2))

U2 = similar(X2)

```

```

V2 = similar(Y2)

for i in eachindex(X2)
    U2[i], V2[i] = f_interaction(X2[i], Y2[i])
end

quiver(
    vec(X2), vec(Y2),
    quiver=(vec(U2), vec(V2)),
    xlabel="Bacteria x",
    ylabel="Antibiotic effect y",
    title="Part B: Phase Portrait",
    label=false
)

crit_x = [p[1] for p in interaction_points]
crit_y = [p[2] for p in interaction_points]

scatter!(crit_x, crit_y, label="Critical Points")
savefig("part_b_phase_portrait.png")

```